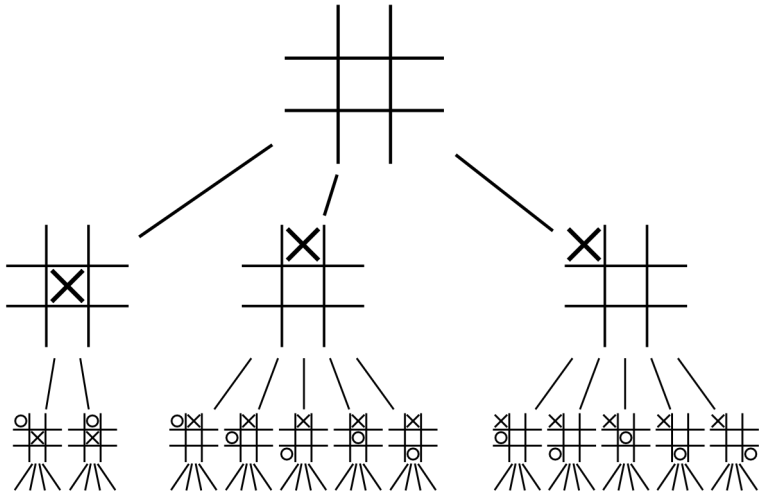
Task 4 Spike: Graphs, Search and Rules

The purpose of this lab is to give you experience representing games as discrete states. A graph of states allows us to explore the outcome of a game by applying actions and to search for game results using graph based search. A sample game graph or game tree for Tic Tac Toe is shown below.



We can treat each possible state the game can be in as a node or vertex on the tree, and each move is an edge that moves from one node to another. Our AIs can then evaluate moves by searching the tree to see where their moves and potential opponent moves may take them.

Knowledge and Skill Gap

Developers of game AI need to know how to represent games and graphs and identify the best move in the current game state by searching the graph. The developer needs to understand the pitfalls of random search and how to optimise for both efficiency and effectiveness.

Goals and Deliverables

1. Tic Tac Toe code modified to represent the game state as a graph.
2. An AI that randomly searches the graph.
3. An AI that improves the efficiency of a basic random search.
4. An AI that improves the effectiveness of a basic random search.

Planning Notes

- Look at the picture above. Each node is the state of the Tic Tac Toe game after a move. To use a graph based approach, your AI will need copies of these boards. How can your code pass these to your AI? Or will the AI call a function to obtain the board state after a given move?
- A random search AI may follow pseudocode like:
    1. Get the current state of the board.
    2. Randomly attempt a move, push the resulting board state onto a list.
    3. If the move results in a victory, return the list.
    4. Otherwise, take the last board state and repeat.
- This AI is both stupid and inefficient. It does not account for opponent moves, and it may attempt invalid moves thousands of times before finding a legal or winning one.
- Improving efficiency: A basic improvement is to check if a move is legal before adding it.
- Improving effectiveness: Effectiveness is about beating opponents. Can your search work backward from a winning state? Can you anticipate opponent responses? The gold standard is the minimax algorithm.

Extensions

- Can you assign probabilities to an opponent's moves and use them to block more effectively?
- Can you create an AI that always plays a perfect game? What about a 4x4 board? Or 3x3x3? Or any board variation?
- Can you visualise your AI's decision making? Printing a chart or diagram of the search tree is useful for debugging.

Links for further reading:

- Tic Tac Toe Strategy
- Minimax Example
- Minimax
- xkcd: Tic-Tac-Toe

Submission Requirements

All labs and spikes now follow the updated submission system:

- Save and push all Python code related to this spike to your Bitbucket repository.
- Save and push any diagrams, planning notes, or supporting documents to the repository.
- Upload your **repository link** to Canvas so your tutor can access both your work and your implementation.

Your spike is complete when all required code and documents are in your repository, and your repository link has been uploaded to Canvas.

